WorkRadar PRD

Bikram Laishram · 12 Oct 2026 · Live: work-radar-theta.vercel.app · Code: github.com/BikramSailorBoy/WorkRadar

WorkRadar is one screen that tells an engineering manager, in under a minute, what to work on today, what happened, what needs attention now and who is blocked, without opening Slack, Jira, GitHub or email.

1. Problem statement

Managers rebuild the same picture every morning from four or more tools, and the most important signal is often in the wrong one. A blocker is mentioned in Slack but never flagged in Jira; an escalation arrives by email; a pull request waits on a reviewer who is on leave. Each tool is accurate, but none answers the manager's real question: where is my attention needed today?

2. Target users

Engineering managers and team leads running 5 to 15 people who work across Slack, Jira, GitHub and email, especially on distributed teams where work happens while the manager is offline. Later: tech leads covering for a manager, and directors overseeing several teams.

3. User personas

Persona	Context	Goal	Frustration
Marty McFly, Engineering Manager (primary)	Darb Pay (fictional), HQ in Cologne; 8 engineers in Riyadh, Cairo and Bengaluru; logs in at 09:30 after the team has been working for hours	Unblock the team and decide before standup	Four tools, and blockers hide in chat
Sara Al-Otaibi, Tech Lead (secondary)	Onboarding squad, Riyadh; needs manager decisions to keep a demo date	Get a scope-vs-date decision fast	Requests get lost in the manager's inbox

4. User journey

Step	Today	With WorkRadar
1. Start the day	Opens Slack, Jira, GitHub and email in turn	Opens one screen; four answer tiles sit above the fold
2. Understand	Pieces together what changed overnight	Reads a 60-second AI brief; every line cites its source item
3. Spot risk	Scans for incidents and escalations	Sees urgent items closest to the radar's centre, with the reason for each
4. Unblock	Hunts for who is stuck, often too late	Sees each blocked person and the chain behind the block
5. Act	Writes follow-ups by hand	Assigns, marks done or snoozes from the feed or the item's page; triage mode walks the urgent list

5. Key pain points

- **Context switching:** four or more tools to answer one question.
- **Hidden blockers:** a blocker said only in chat never reaches the tracker.
- **Noise:** bots, CI and chatter bury the few events that matter.
- **Lost decisions:** requests aimed at the manager sit unanswered.
- **Time zones:** a distributed team's day happens while the manager is offline.

6. Success metrics

Metric	Target	Status
Planted test situations surfaced in the right panel	10 of 11 minimum	11 of 11
Noise items in Top work or Needs attention	0	0
Brief sentences with a valid source citation	100%	100%, enforced in code
Live AI brief generated	Under 8 s	3.6 to 4 s
Pilot: time to answer the four questions	Under 60 s	To measure
Pilot: median blocker age, and triage completion the same day	Falling week on week	To measure

7. MVP scope

In: dashboard with the four answers on one screen; AI-generated summary; priority feed; search; a detail page per item; Assign, Mark Done and Snooze with undo; radar with an explainable 0 to 100 score; AI reasons, Ask and

nudges calling Claude live; demo mode when no API key is set; mock data with 12 planted test situations.

Out: live connectors to Slack, Jira, GitHub and email; write-back to those tools; sign-in; multi-team views.

8. Assumptions

- Mock event data is a fair stand-in for connector webhooks; the replay-based design maps directly onto them.
- One manager, one team; working weeks differ by location (Friday and Saturday off in Riyadh, Saturday and Sunday in Cologne and Bengaluru).
- Managers will trust AI only if every claim cites its source, so rules rank and AI explains.
- The product ranks work, never people.

9. Features

Feature	Priority	Why
Dashboard: four answer tiles that expand on click	Must-have	The brief's core ask, readable in 10 seconds
Priority feed: every open item ranked, with filters	Must-have	One ordered list of what to do next
Search across items, people and message text	Must-have	Find anything without opening four tools
Detail page per item, with its full cross-tool timeline	Must-have	Shareable context behind every ranking
Basic actions: Assign, Mark Done, Snooze, with undo	Must-have	Act where the insight is
Radar: distance shows urgency, sector shows source tool	Must-have	Shows the shape of the day at a glance
Explainable priority score with a reason on every item	Must-have	Rules rank, so the ranking is auditable
AI-generated summary (morning brief) with validated citations	Must-have	Summary the manager can trust and check
Blocker detection, including blockers mentioned only in Slack or email	Must-have	Finds what the trackers miss
Live AI with automatic, labelled demo fallback	Must-have	Works for any reviewer, with or without a key
Triage mode with AI-drafted nudges (built)	Nice-to-have	Fast pass through urgent items
Day replay with track trails (built)	Nice-to-have	Makes "what happened" visible
Ask WorkRadar, with answers from the data only (built)	Nice-to-have	Ad hoc questions without new screens
Noise shown as suppressible clutter (built)	Nice-to-have	Makes the filtering visible
Write-back, a brief pushed to Slack, a director view	Later	Needs live connectors first

The full product spec (scoring rules, AI design, options considered and risks) is in the repo as WorkRadar-Product-Spec.pdf.